0261. Graph Valid Tree

Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/graph-v

Problem

- N nodes and undirected edges: do they form a valid tree (connected and acyclic)?

Intuition

- A tree with n nodes has exactly n - 1 edges. With that edge count, "no cycle" is equivalent to "connected".
- So: check edges == n - 1, then Union-Find to make sure no edge closes a cycle.

Brute Force

- DFS checking for cycles (track parent) and then visited count == n.

Optimal Approach

- If edges.length != n - 1 return false.
- Union every edge; if any union finds both ends already connected, return false.
- Return true.

Worked Example

```
n = 5, edges [[0,1],[1,2],[2,3],[1,3],[1,4]].

edges = 5 != n - 1 = 4 -> false (cycle 1-2-3)
```

Java Solution

```java
public class GraphValidTree {
    public boolean validTree(int n, int[][] edges) {
        if (edges.length != n - 1) return false;
        int[] parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
        for (int[] e : edges) {
            int a = find(parent, e[0]), b = find(parent, e[1]);
            if (a == b) return false;
            parent[a] = b;
        }
        return true;
    }

    private int find(int[] p, int x) {
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    }
}
```

Complexity

- Time: O(n alpha(n)). Space: O(n).

Pitfalls

- N = 1 with no edges is a valid tree.